

الجمهورية الجزائرية الديمقراطية الشعبية

Democratic and Popular Republic of Algeria

وزارة التعليم العالي والبحث العلمي

Ministry of Higher Education and Scientific Research

المركز الجامعي افلو - الشريف بوشوشة

Aflou University Center

Faculty of Sciences

Department of Computer Science



End of Study Dissertation

To obtain the diploma of Bachelor's degree in Computer Science

Development of a Hybrid Encryption System

Directed by: Ahlem Chachi

Supported on: May 31, 2026

Before the jury composed of:

President:

Examiner:

Framer: Dr. Azouzi Oussama

Academic year: 2025 / 2026

Dedication

إن الحمد لله نحمده ونستعينه ونستهديه ونعوذ بالله من شرور أنفسنا وسيئات أعمالنا، من يهده الله فهو المهتدي، ومن يضلل فلا هادي له، وأشهد أن لا إله إلا الله وأن محمداً عبده ورسوله.

To my dear mother, I dedicate my graduation to the one who guided me on my life's path, who instilled in me a sense of self-worth, and whose prayers have always surrounded me.

To my dear father, I dedicate my graduation to the light that illuminated my path, to my first teacher, and to the beloved one whose name I have had the honor of bearing.

To my advisor, Dr. Oussama Azouzi, who instilled in me a passion for knowledge and research

To my family and friends, and to everyone who stood by my side and left a lasting impression on my heart.

أسأل الله العظيم أن يجعل هذا العمل بداية خير في حياتي، وأن يوفقني إلى ما فيه صلاح ديني ودنياي. والحمد لله رب العالمين.

Abstract

This project presents the design and implementation of a hybrid encryption desktop application named *Crypto App*. The system integrates AES-128 in EAX mode for high-speed symmetric encryption with RSA-2048 using OAEP padding for secure asymmetric key encapsulation. The application supports five data types: text messages, images, files, videos, and RSA key management.

It is built entirely in Python using PyCryptodome for cryptographic operations, CustomTkinter for the modern graphical user interface, and Pillow for image processing. Each encryption operation generates a fresh AES session key, ensuring forward secrecy. The AES-EAX authentication tag guarantees data integrity throughout all operations.

Performance is monitored in real time via progress bars and elapsed time displays. All long-running operations execute in background threads to maintain full interface responsiveness.

Keywords: AES, RSA, hybrid encryption, EAX, OAEP, PyCryptodome, CustomTkinter, Python, data security, cryptography.

ملخص

يقدم هذا المشروع تصميم وتنفيذ تطبيق سطح مكتب للتشفير الهجين يحمل اسم App. Crypto. يدمج النظام خوارزمية التشفير المتماثل AES-128 في وضع EAX لتحقيق سرعة عالية في التشفير، مع خوارزمية التشفير غير المتماثل RSA-2048 باستخدام حشو OAEP لتغليف مفتاح الجلسة بشكل آمن.

يدعم التطبيق خمسة أنواع من البيانات: الرسائل النصية، الصور، الملفات، مقاطع الفيديو، وإدارة مفاتيح RSA. تم بناؤه بالكامل بلغة Python باستخدام مكتبة PyCryptodome للعمليات التشفيرية، ومكتبة CustomTkinter لواجهة المستخدم الرسومية الحديثة، ومكتبة Pillow لمعالجة الصور.

تُولد مفتاح جلسة AES جديد لكل عملية تشفير لضمان السرية التامة للأمام. يضمن وسم المصادقة الخاص بوضع AES-EAX سلامة البيانات طوال جميع العمليات. تتم مراقبة الأداء في الوقت الفعلي عبر أشرطة التقدم وعروض الوقت المنقضي. تُنفَّذ جميع العمليات الطويلة في خيوط خلفية للحفاظ على استجابة الواجهة الكاملة.

الكلمات المفتاحية: AES، RSA، التشفير الهجين، EAX، OAEP، PyCryptodome، CustomTkinter، Python، أمن البيانات، التشفير.

Contents

2	Dedication	
	Abstract	3
4	ملخص	
	List of Acronyms	8
	General Introduction	9
1	Background and Related Works	11
1.1	Symmetric Encryption	11
1.1.1	Definition: Symmetric Encryption	11
1.1.2	Advanced Encryption Standard (AES)	12
1.1.3	AES Modes of Operation	12
1.2	Asymmetric Encryption	13
1.2.1	Definition: Asymmetric Encryption	13
1.2.2	RSA Algorithm	13
1.2.3	OAEP Padding	14
1.3	Hybrid Encryption	14
1.3.1	Definition: Hybrid Encryption	14
1.4	Related Work	15
1.5	Motivation for the Project	18
2	Methodology and Implementation	19
2.1	Methodology	19
2.1.1	System Architecture	19
2.1.2	Tools and Libraries	20
2.2	Implementation	21
2.2.1	Application Design and Technologies	21
2.2.2	Application Screenshots	21
2.2.3	UML Diagrams	25
2.2.4	Hybrid Encryption Scheme	27

2.2.5	Key Management	27
2.2.6	Implementation Details	27
2.2.7	Testing and Validation	29
General Conclusion		30
	Summary of Work	30
	Achievements	30
	Limitations	31
	Future Work	31
	Concluding Remarks	32
Bibliography		33

Contents

List of Tables

1.1	Comparison of AES Modes of Operation	12
1.2	Comparison of Related Works with Our Crypto App	17
2.1	Summary of Tools and Libraries Used	20
2.2	Design Decisions and Their Justifications	21

List of Acronyms

AES	Advanced Encryption Standard
API	Application Programming Interface
CBC	Cipher Block Chaining
EAX	Encrypt-then-Authenticate-then-Translate
ECB	Electronic Codebook
GCM	Galois/Counter Mode
GUI	Graphical User Interface
NIST	National Institute of Standards and Technology
OAEP	Optimal Asymmetric Encryption Padding
PEM	Privacy Enhanced Mail
RSA	Rivest–Shamir–Adleman
TLS	Transport Layer Security

General Introduction

In today's digital world, the protection of sensitive data has become a fundamental requirement across all sectors. The rapid growth of digital communication and data exchange has exposed individuals and organizations to increasing threats of unauthorized access, data breaches, and information theft. Cryptography provides the essential tools to address these threats by ensuring data confidentiality, integrity, and authenticity.

Problem Statement

Two primary categories of cryptographic algorithms are widely used:

- **Symmetric encryption** (e.g., AES): Relies on a single shared key for both encryption and decryption. It offers high performance for large data volumes but suffers from the challenge of secure key distribution.
- **Asymmetric encryption** (e.g., RSA): Employs a mathematically linked key pair (public and private). It solves the key distribution problem but is computationally intensive and unsuitable for directly encrypting large amounts of data.

Proposed Solution

This project addresses these limitations by combining both approaches into a unified hybrid encryption system implemented as a fully functional desktop application named

Crypto App. The system integrates:

- **AES-128 in EAX mode** for high-speed symmetric encryption
- **RSA-2048 with OAEP padding** for secure asymmetric key encapsulation
- **EAX authentication tag** for data integrity verification

Objectives

The primary objectives of this project are:

1. Design and implement a hybrid encryption system combining AES-128 (EAX mode) and RSA-2048 (OAEP padding)

-
2. Develop a desktop application with a modern, intuitive graphical user interface using CustomTkinter
 3. Support encryption and decryption of five data types: text messages, images, files, videos, and RSA keys
 4. Implement secure RSA key generation, display, and export in PEM format
 5. Provide real-time performance feedback through progress bars and elapsed time measurements
 6. Ensure data integrity using the AES-EAX authentication tag mechanism

Report Structure

This report is organized as follows: after this general introduction, **Chapter 1** provides theoretical background on symmetric and asymmetric encryption, hybrid cryptography, and related work. **Chapter 2** describes the methodology, system architecture, and full implementation details of the *Crypto App* application. Finally, the **General Conclusion** summarizes achievements, discusses limitations, and proposes future improvements.

Chapter 1

Background and Related Works

This chapter presents the theoretical foundations upon which the *Crypto App* application is built. It covers the two main categories of cryptographic algorithms used in this project, namely symmetric encryption and asymmetric encryption, followed by an explanation of the hybrid encryption model that combines both.

1.1 Symmetric Encryption

1.1.1 Definition: Symmetric Encryption

Symmetric Encryption

Meaning: A type of encryption where the **same key** is used for both encryption and decryption. Like a lockbox with one key that both locks and unlocks it.

Main Components:

- **Single Secret Key:** Used for both operations
- **Encryption Algorithm:** Such as AES or DES
- **Plaintext:** Original data before encryption
- **Ciphertext:** Encrypted data after encryption

Real-world Example: A standard door lock where the same key locks and unlocks the door.

In Our Project: We use AES-128 to encrypt the actual data (messages, images, videos).

1.1.2 Advanced Encryption Standard (AES)

AES (Advanced Encryption Standard)

Meaning: A symmetric encryption algorithm that is the global standard since 2001. It divides data into 128-bit blocks and encrypts each block.

Main Components:

- **Block Size:** 128 bits (16 bytes)
- **Key Sizes:** 128, 192, or 256 bits
- **Rounds:** 10 rounds for AES-128
- **Internal Operations:** SubBytes, ShiftRows, MixColumns

Real-world Example: Used in ZIP file encryption, Wi-Fi (WPA2), WhatsApp messages.

In Our Project: We use **AES-128** (128-bit key) to encrypt all data types at high speed.

1.1.3 AES Modes of Operation

EAX Mode (Encrypt-then-Authenticate-then-Translate)

Meaning: An AES operation mode that provides **encryption and authentication together**. It ensures that data cannot be read (encryption) and cannot be tampered with (authentication).

Main Components:

- **Encryption:** Converts data into unreadable form
- **Authentication Tag:** A small value added to verify data integrity
- **Nonce:** A random number used once to prevent repeated outputs

Real-world Example: Like sending a letter in a sealed envelope (encryption) with a wax seal (authentication). If the seal is broken, you know someone tampered with it.

In Our Project: EAX ensures that encrypted files cannot be tampered with. The authentication tag is verified during decryption.

Mode	Description	Authentication
ECB	Each block encrypted independently	No
CBC	Blocks chained with XOR	No
GCM	Counter mode + authentication	Yes
EAX	Encryption + Authentication (Used in our project)	Yes

Table 1.1: Comparison of AES Modes of Operation

1.2 Asymmetric Encryption

1.2.1 Definition: Asymmetric Encryption

Asymmetric Encryption

Meaning: A type of encryption that uses **two different keys**: a public key for encryption and a private key for decryption. Like a mailbox where anyone can drop a letter (using the public key), but only the owner can open it (using the private key).

Main Components:

- **Public Key:** Shared with everyone, used only for encryption
- **Private Key:** Kept secret by the owner, used only for decryption
- **Mathematical Algorithm:** Based on one-way functions (e.g., factoring large numbers)

Real-world Example: HTTPS protocol on websites, Bitcoin, digital signatures.

In Our Project: We use RSA-2048 to encrypt only the AES session key (not the large data).

1.2.2 RSA Algorithm

RSA (Rivest-Shamir-Adleman)

Meaning: The most famous asymmetric encryption algorithm, invented in 1977. Its security relies on the **difficulty of factoring large numbers into their prime factors**.

Main Components:

- **Two Large Primes (p, q):** The basis for key generation
- **Modulus $n = p \times q$:** Part of both public and private keys
- **Public Exponent (e):** Usually 65537
- **Private Exponent (d):** Calculated from e, p, and q

Simple Math Example: If $p=61$ and $q=53$, then $n=3233$. It is very hard to find 61 and 53 if you only know 3233.

In Our Project: We use **RSA-2048** (n is 2048 bits) to encrypt the AES key, providing security equivalent to 112-bit symmetric encryption.

1.2.3 OAEP Padding

OAEP (Optimal Asymmetric Encryption Padding)

Meaning: A padding scheme added to messages before RSA encryption. It adds **randomness** so that the same message produces different outputs each time.

Main Components:

- **Random Seed:** Used to generate a random mask
- **Mask:** Mixes with the message to make it random
- **Fixed Prefix:** Added to verify decryption integrity

Example: Without OAEP, if you encrypt "YES" 100 times, you get 100 identical results. With OAEP, each result is completely different.

In Our Project: We use OAEP with RSA-2048 to prevent deterministic attacks and ensure that the same encrypted AES key looks different each time.

1.3 Hybrid Encryption

1.3.1 Definition: Hybrid Encryption

Hybrid Encryption

Meaning: Combining symmetric and asymmetric encryption to benefit from the **speed** of symmetric and the **secure key management** of asymmetric encryption.

Main Components:

- **Session Key:** A random key used only once (AES-128)
- **AES-encrypted Data:** Using the session key
- **RSA-encrypted Session Key:** Using the recipient's public key

How It Works (Step by Step):

1. Generate a random AES session key
2. Encrypt the data with the random key → ciphertext
3. Encrypt the random key with the RSA public key → encrypted key
4. Send (ciphertext + encrypted key) together

Real-world Example: TLS/SSL protocol used in HTTPS (websites), PGP for email encryption.

In Our Project: AES-128 encrypts the actual data, RSA-2048 encrypts the AES key, and both are saved in a single binary file.

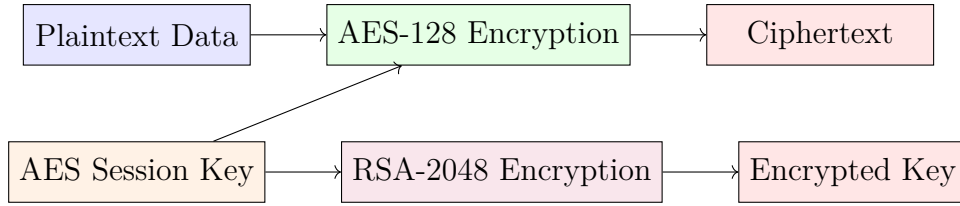


Figure 1.1: Hybrid Encryption Process Overview

1.4 Related Work

The combination of AES and RSA in hybrid encryption systems has been extensively studied and validated in the literature. [1] provides a comprehensive treatment of cryptographic principles and highlights the practical advantages of hybrid schemes.

Research in secure file transfer, email encryption, and cloud storage consistently demonstrates that AES-RSA hybrid systems offer an optimal trade-off between performance and security. The use of authenticated encryption modes such as EAX and GCM has been increasingly recommended to provide simultaneous confidentiality and integrity guarantees [2].

Several existing applications and libraries implement hybrid encryption using similar cryptographic principles. This section reviews notable examples that are closely related to our *Crypto App* project.

Dual-Encryption Integrative RSA-AES Encryption

This chat application [7] implements dual-layer encryption where files are first encrypted with AES, then the AES key is encrypted with RSA before transmission or storage. This approach is methodologically identical to our project’s file encryption scheme. The application demonstrates the same core principle: combining the speed of symmetric encryption with the security of asymmetric key exchange. However, it is designed primarily for messaging scenarios and does not offer the multi-type data support (images, videos, files) or the modern tabbed interface with visual preview features available in our *Crypto App*.

Ferrocrypt

Ferrocrypt [8] is a pure Rust encryption tool that provides both symmetric (XChaCha20-Poly1305) and hybrid (RSA-4096) encryption modes through CLI and GUI interfaces. It supports file and directory encryption with a focus on performance and security. Unlike our application, Ferrocrypt does not provide a built-in key management interface or support for image preview, which are distinctive features of our *Crypto App*.

CryptoEngine

CryptoEngine is a desktop application built with Python and PyQt6 that implements RSA + AES hybrid encryption. It allows users to encrypt data using SSH-compatible RSA public keys and decrypt using private keys. The application provides a tabbed interface for key management and encryption operations. Similar to our project, it uses the hybrid encryption approach where AES encrypts the actual data and RSA protects the AES key. However, CryptoEngine focuses primarily on text and file encryption without specialized support for images or videos with visual preview, which is a key feature of our *Crypto App*.

Cross Crypto TS

Cross Crypto TS [6] is a multi-language library (supporting TypeScript, Python, and Rust) that provides interoperable hybrid encryption using AES-GCM and RSA-OAEP. It supports JSON data, binary files, and streaming mode for large files. The library follows the same cryptographic principles as our project: AES for symmetric encryption and RSA for key encapsulation with OAEP padding. However, Cross Crypto TS is a library rather than a complete desktop application, and it lacks a graphical user interface. Our *Crypto App* extends these concepts by providing an accessible, user-friendly interface with real-time progress feedback.

Comparison Summary

While these existing projects share similar cryptographic foundations with our work, our *Crypto App* distinguishes itself through the following contributions:

- **Multi-type Data Support:** Unified interface for text messages, images, files, and videos within a single application.
- **Visual Feedback:** Side-by-side image preview and real-time progress bars with elapsed time display.
- **Modern Interface:** Intuitive five-tab layout built with CustomTkinter, offering a professional user experience.
- **Integrated Key Management:** Built-in RSA key generation, display, and export functionality in PEM format.
- **Responsive Design:** Background threading ensures the GUI remains responsive during long operations.

Comparison Table

Feature / Application	CryptoEngine	Cross Crypto TS	Dual-Encryption	Crypto App (Ours)
Platform	Desktop (PyQt6)	Library (Multi-language)	Chat Application	Desktop (CustomTkinter)
GUI Available	Yes	No	Yes	Yes
Text Encryption	Yes	Yes	Yes	Yes
Image Encryption	No	Yes	No	Yes
Image Preview	No	No	No	Yes (Side-by-side)
Video Encryption	No	No	No	Yes
Progress Bar	No	No	No	Yes
Elapsed Time Display	No	No	No	Yes
RSA Key Management	Yes	Yes	No	Yes
PEM Export	Yes	Yes	No	Yes
Background Threading	No	Yes	No	Yes

Table 1.2: Comparison of Related Works with Our Crypto App

Contributions of Our Work

Based on the comparison above, our *Crypto App* distinguishes itself through the following contributions:

- **Multi-type Data Support:** Unified interface for text messages, images, files, and videos within a single application.
- **Visual Feedback:** Side-by-side image preview and real-time progress bars with elapsed time display.
- **Modern Interface:** Intuitive five-tab layout built with CustomTkinter, offering a professional user experience.
- **Integrated Key Management:** Built-in RSA key generation, display, and export functionality in PEM format.
- **Responsive Design:** Background threading ensures the GUI remains responsive during long operations.

1.5 Motivation for the Project

Despite the availability of cryptographic libraries, there remains a lack of accessible, user-friendly desktop applications that demonstrate hybrid encryption across multiple data types in a unified interface. This project bridges the gap between theoretical cryptographic concepts and practical implementation, providing a tool that is both educational and functional.

Chapter 2

Methodology and Implementation

This chapter describes the design decisions, system architecture, and full implementation details of the *Crypto App* application. It covers the hybrid encryption scheme adopted, the key management strategy, the technology stack used, and the structure of each functional component of the application. Code excerpts are provided to illustrate the core cryptographic operations.

2.1 Methodology

2.1.1 System Architecture

The application *Crypto App* follows a layered architecture:

Layered Architecture

Meaning: A design pattern that separates the application into distinct layers, each with a specific responsibility.

Three Layers in Our Project:

1. **Cryptographic Layer:** Handles all encryption and decryption operations using AES-EAX and RSA-OAEP via PyCryptodome.
2. **File Management Layer:** Reads, writes, and organizes encrypted binary output files.
3. **Presentation Layer:** The graphical user interface built with CustomTkinter, organized into a home screen and five functional tabs.

2.1.2 Tools and Libraries

Tool/Library	Version	Purpose
Python	3.13	Main programming language
PyCryptodome	Latest	Cryptographic operations (AES, RSA)
CustomTkinter	5.2.2	Modern graphical user interface
Pillow (PIL)	Latest	Image processing and display
NumPy	Latest	Numerical operations
PyInstaller	Latest	Creating standalone executable
OS Module	Built-in	File system operations
Threading	Built-in	Background task execution

Table 2.1: Summary of Tools and Libraries Used

PyCryptodome

Description: PyCryptodome is a self-contained Python package of low-level cryptographic primitives.

Used for:

- AES key generation: `Crypto.Random.get_random_bytes(16)`
- RSA key generation: `Crypto.PublicKey.RSA.generate(2048)`
- AES encryption/decryption: `Crypto.Cipher.AES`
- RSA encryption/decryption: `Crypto.Cipher.PKCS1_OAEP`

CustomTkinter 5.2.2

Description: CustomTkinter is a modern Python UI library built on top of Tkinter.

Used for:

- Main window: `CTk()`
- Buttons: `CTkButton`
- Labels: `CTkLabel`
- Text input: `CTkTextbox`
- Tab view: `CTkTabview`
- Progress bar: `CTkProgressBar`

Pillow (PIL)

Description: Pillow is a Python Imaging Library for image processing.

Used for:

- Opening images: `Image.open()`
- Converting images for display: `CTkImage()`
- Side-by-side image preview

2.2 Implementation

2.2.1 Application Design and Technologies

Application Layout

Main Screen Elements:

1. **Top Navigation Bar:** Contains Home button and status indicator showing "AES-128 + RSA-2048 Active"
2. **Tab View:** Five tabs for different data types
3. **Content Area:** Each tab contains specific controls
4. **Status Area:** Progress bars and elapsed time display

The Five Tabs:

- **Message Tab:** Text input field, Encrypt/Decrypt buttons, result display
- **Image Tab:** File selector, preview area, side-by-side image display
- **File Tab:** File selector, Encrypt/Decrypt buttons, status messages
- **Video Tab:** File selector, progress bar, elapsed time label
- **Keys Tab:** Generate Keys button, PEM display area, Export buttons

Decision	Reason
CustomTkinter instead of Tkinter	Modern look, better user experience
Five separate tabs	Organized interface, easy navigation
Background threading	GUI remains responsive during encryption
Progress bars	User knows operation status
Side-by-side image preview	Visual confirmation of lossless encryption
PEM format for keys	Standard format, compatible with other tools

Table 2.2: Design Decisions and Their Justifications

2.2.2 Application Screenshots

This section presents the graphical user interface of the *Crypto App* application.

Home Screen



Figure 2.1: Home Screen of Crypto App

Message Tab

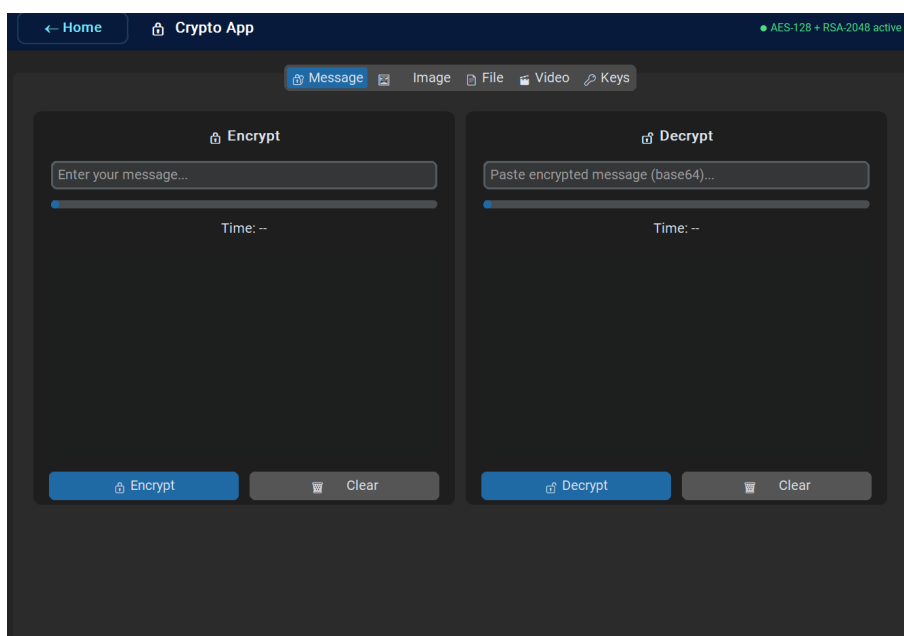


Figure 2.2: Message Tab - Text Encryption and Decryption

Image Tab

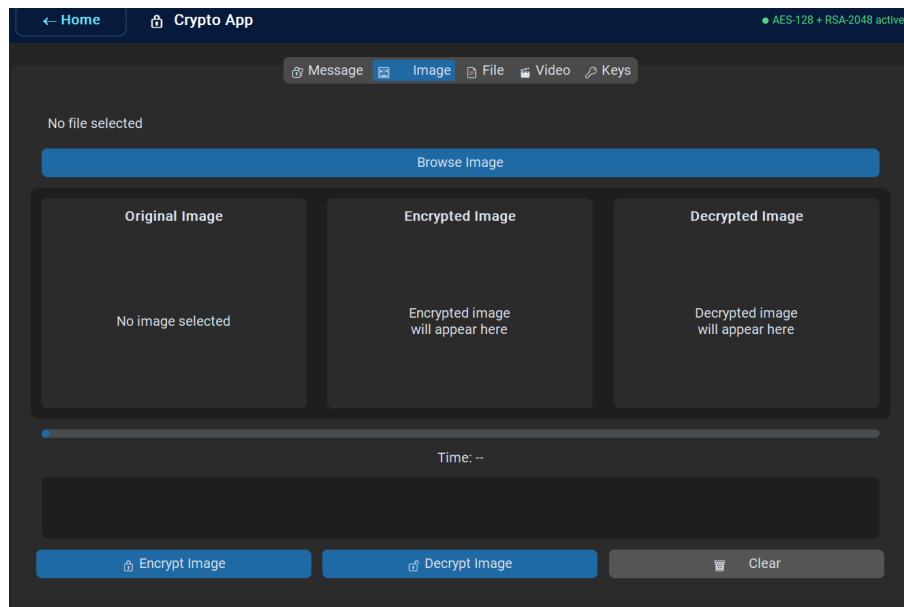


Figure 2.3: Image Tab - Image Encryption with Preview

File Tab

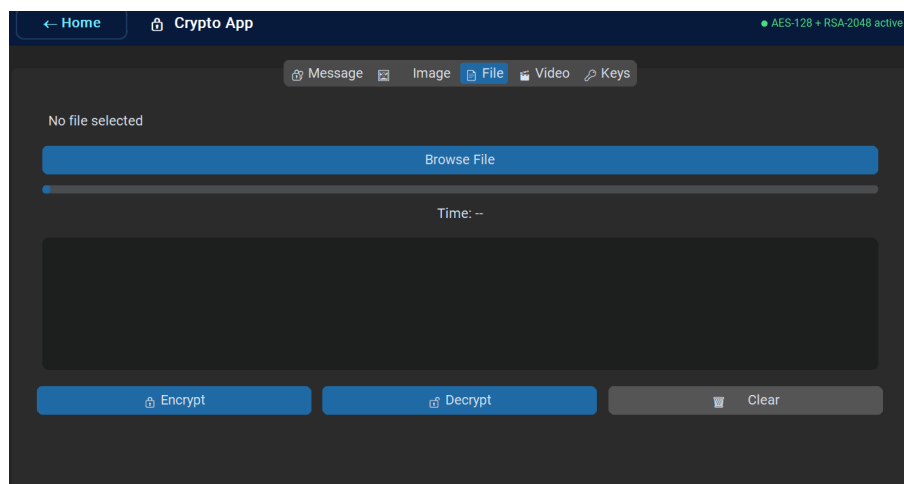


Figure 2.4: File Tab - Generic File Encryption and Decryption

Video Tab

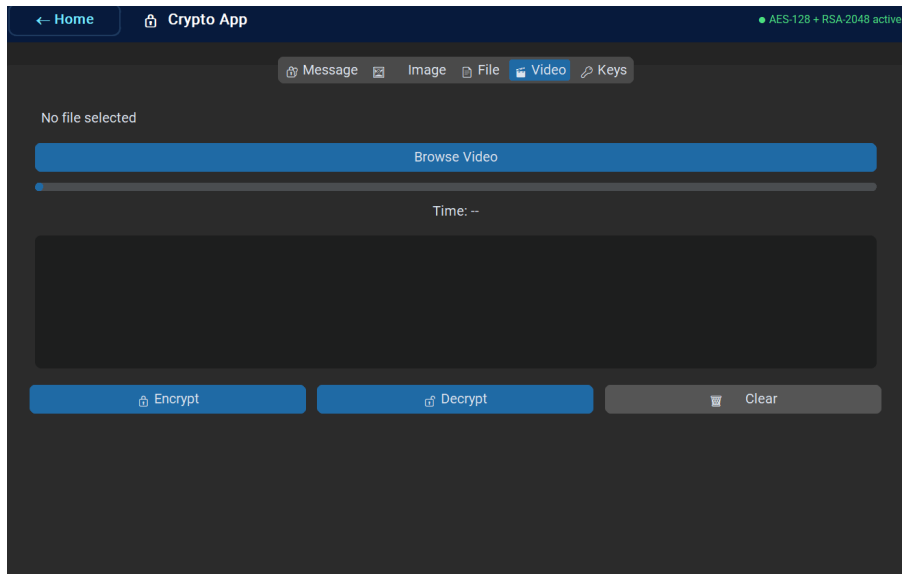


Figure 2.5: Video Tab - Video Encryption with Progress Tracking

Keys Tab

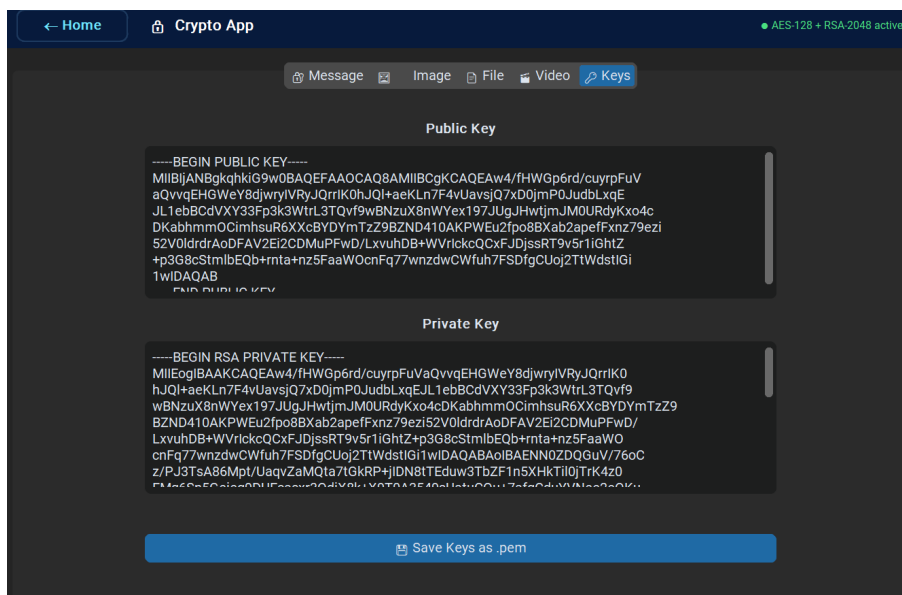


Figure 2.6: Keys Tab - RSA Key Management

2.2.3 UML Diagrams

Use Case Diagram

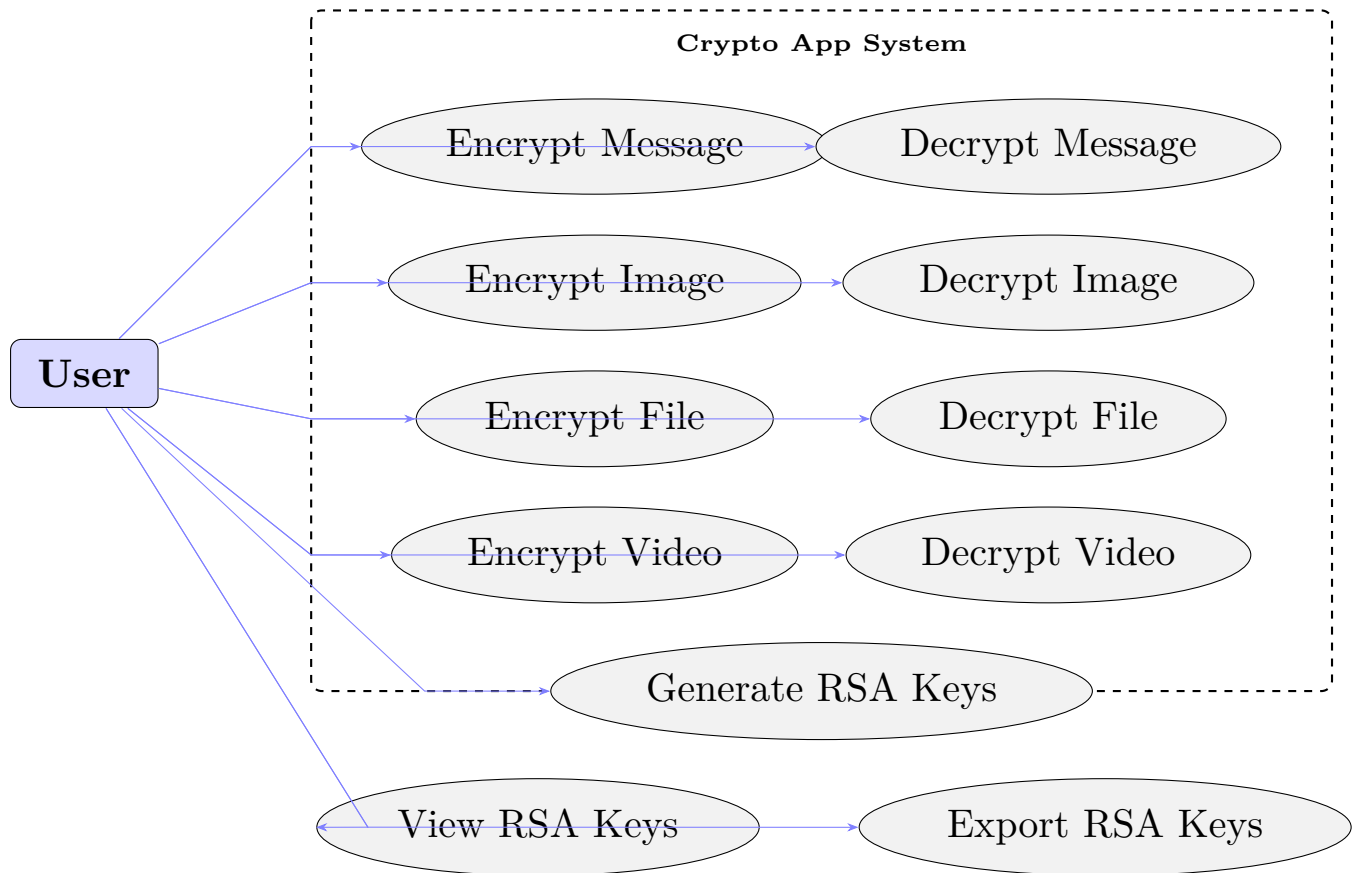


Figure 2.7: Use Case Diagram for Crypto App System

Activity Diagram

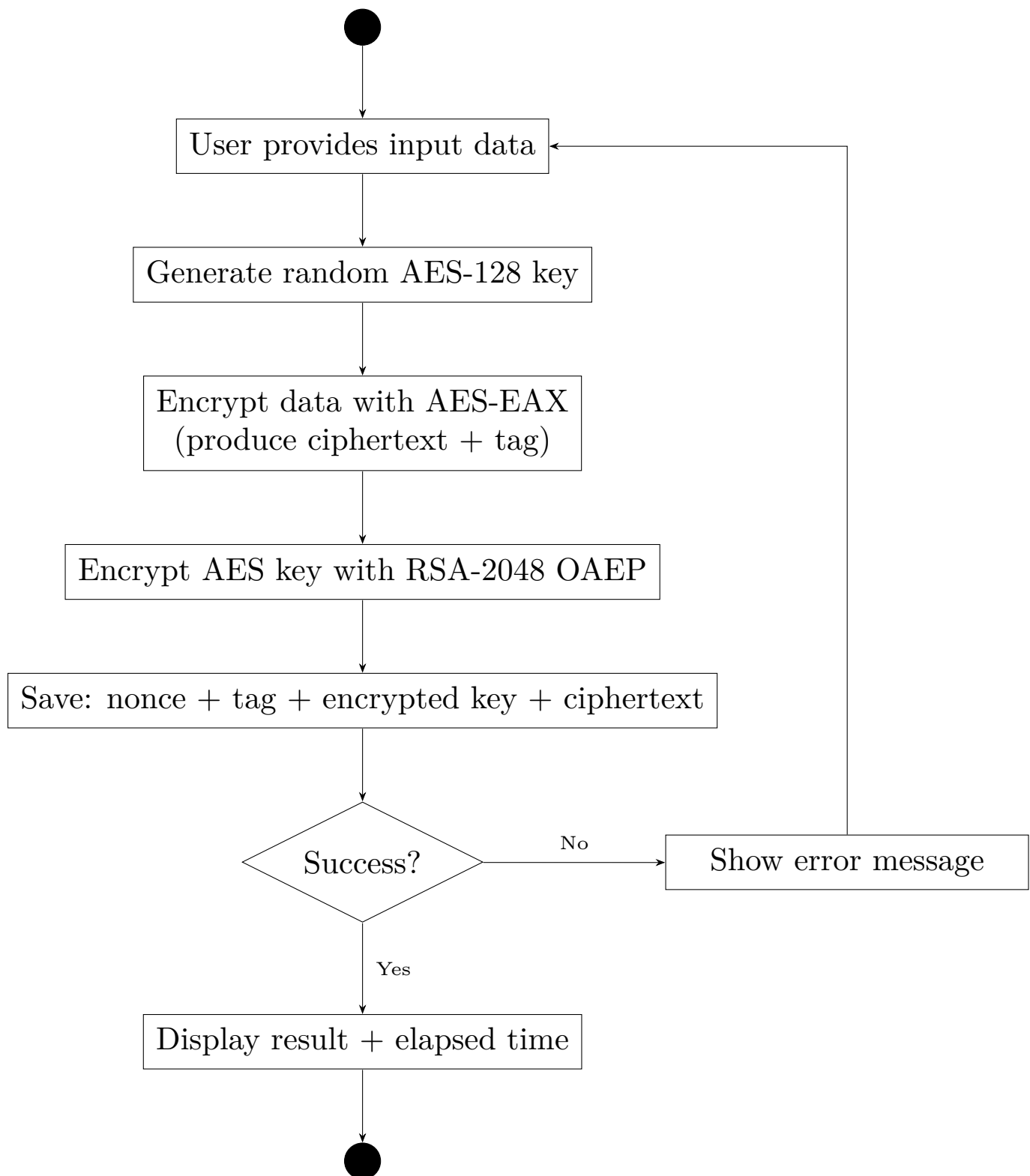


Figure 2.8: Activity Diagram for Hybrid Encryption Process

2.2.4 Hybrid Encryption Scheme

Encryption Process

Encryption Process (Step by Step)

1. Generate a random 128-bit AES session key using `get_random_bytes(16)`
2. Generate a random nonce automatically by AES-EAX mode
3. Encrypt the plaintext data using AES-EAX → produces ciphertext + authentication tag
4. Encrypt the AES session key using RSA-2048 public key with OAEP padding
5. Save all components in a single binary file: nonce + tag + encrypted key + ciphertext

Decryption Process

Decryption Process (Step by Step)

1. Read the binary file to extract: nonce, tag, encrypted AES key, ciphertext
2. Decrypt the AES session key using RSA private key
3. Decrypt the ciphertext using the recovered AES key and nonce
4. Verify the authentication tag to confirm data integrity before accepting the output

2.2.5 Key Management

Key Management

Meaning: The process of generating, storing, and distributing cryptographic keys securely.

In Our Project:

- RSA key pairs (2048 bits) are generated at application startup using Py-Cryptodome's cryptographically secure random number generator
- The Keys tab allows users to view public/private keys in PEM format and export them to `.pem` files
- A **fresh AES session key** is generated for every encryption operation → ensures forward secrecy and prevents key reuse attacks

Forward Secrecy: If a long-term key is compromised, past session keys remain secure because each session uses a new random key.

2.2.6 Implementation Details

RSA Key Generation

Listing 2.1: RSA Key Generation at Startup

```
1 from Crypto.PublicKey import RSA
2 from Crypto.Random import get_random_bytes
3
4 key = RSA.generate(2048)
5 private_key = key
6 public_key = key.publickey()
7 aes_key = get_random_bytes(16)
```

Message Encryption

Listing 2.2: Hybrid Message Encryption

```
1 from Crypto.Cipher import PKCS1_OAEP, AES
2
3 cipher_aes = AES.new(aes_key, AES.MODE_EAX)
4 nonce = cipher_aes.nonce
5 ciphertext, tag = cipher_aes.encrypt_and_digest(message)
6
7 cipher_rsa = PKCS1_OAEP.new(public_key)
8 encrypted_aes_key = cipher_rsa.encrypt(aes_key)
```

Binary File Structure

Binary File Structure

For images, files, and videos, the encrypted output is stored as a structured binary file:

Bytes	Content
16 bytes	AES nonce
16 bytes	AES-EAX authentication tag
256 bytes	RSA-encrypted AES session key
Remaining	AES-encrypted data payload

Threading and Responsiveness

Threading

Meaning: A technique that allows multiple tasks to run concurrently. Long-running operations run in background threads so the GUI doesn't freeze.

In Our Project: All encryption/decryption operations run in daemon background threads, and progress bars update incrementally to reflect the current processing stage.

2.2.7 Testing and Validation

The system was tested across all supported data types and file sizes. Each test verified that the decrypted output was byte-for-byte identical to the original input. The integrity check in the Message tab explicitly compares the decrypted text with the original input and reports a PASSED or FAILED status. Authentication tag verification correctly detects any modification of the ciphertext.

General Conclusion

Summary of Work

This project successfully designed and implemented *Crypto App*, a hybrid encryption desktop application combining AES-128 in EAX mode with RSA-2048 using OAEP padding. The application provides a unified, modern interface for encrypting and decrypting text messages, images, files, and videos, along with a dedicated tab for RSA key management.

The hybrid scheme delivers both the computational efficiency of symmetric encryption and the secure key management of asymmetric encryption, while the AES-EAX authentication tag guarantees data integrity throughout all operations. The application was built entirely in Python using PyCryptodome for cryptographic operations, CustomTkinter for the graphical user interface, and Pillow for image processing.

Achievements

The following are the main achievements of this project:

- **Hybrid Encryption:** AES-128 in EAX mode combined with RSA-2048 using OAEP padding has been fully implemented and tested.
- **Multi-type Data Support:** The application supports encryption and decryption of five data types: text messages, images, files, and videos.
- **Data Integrity:** The AES-EAX authentication tag guarantees data integrity by detecting any tampering with the ciphertext during decryption.
- **Key Management:** RSA key pairs can be generated, displayed in PEM format, and exported to external files for backup or sharing.
- **Real-time Performance Feedback:** Progress bars and elapsed time displays provide users with immediate feedback during long operations.
- **Responsive User Interface:** Background threading ensures all long-running cryp-

tographic operations do not freeze the GUI, maintaining a smooth user experience.

- **Modern Interface Design:** A professional home screen, top navigation bar, and organized five-tab layout make the application intuitive and easy to use.
- **Visual Image Preview:** The Image tab displays original and decrypted images side by side, providing visual confirmation of lossless encryption.
- **Video Progress Tracking:** The Video tab includes a progress bar and elapsed time counter for monitoring encryption of large video files.

Limitations

Despite the successful implementation, the current version of *Crypto App* has some limitations:

- **Persistent Key Storage:** RSA keys are regenerated on every application restart. Keys are not saved between sessions, requiring users to regenerate or manually export/import them.
- **Single File Processing:** The system processes one file at a time. Batch encryption of multiple files is not supported.
- **User Authentication:** No user authentication or access control mechanism is implemented. Anyone with access to the application can use it.
- **Operating System:** The application is currently limited to the Windows operating system. Linux and macOS versions are not yet available.
- **Key Size Options:** Only RSA-2048 and AES-128 are supported. Users cannot choose higher security levels like RSA-4096 or AES-256.

Future Work

Based on the limitations identified, the following improvements are proposed for future versions:

- **Persistent Key Storage:** Implement password-protected saving and loading of RSA key pairs across sessions, allowing users to reuse their keys.
- **Digital Signatures:** Integrate RSA digital signatures to verify sender authenticity and ensure non-repudiation of encrypted messages.
- **Batch Encryption:** Add support for encrypting and decrypting multiple files simultaneously to improve productivity.
- **Larger Key Sizes:** Extend support to AES-256 and RSA-4096 for users with

higher security requirements.

- **Cross-platform Support:** Port the application to Linux and macOS using platform-specific packaging tools.
- **Cloud Integration:** Enable secure encrypted file sharing through cloud storage services like Google Drive or Dropbox.
- **Mobile Application:** Develop a mobile version of Crypto App for iOS and Android devices.
- **File Compression:** Add compression before encryption to reduce output file sizes.

Concluding Remarks

In conclusion, *Crypto App* successfully demonstrates the practical implementation of hybrid encryption concepts. It provides an accessible, user-friendly tool that bridges the gap between theoretical cryptography and real-world application. The system achieves all initial objectives, including support for multiple data types, real-time feedback, and data integrity verification.

The project serves as a solid foundation for future enhancements and can be extended to meet more advanced security requirements. It also serves as an educational resource for understanding how hybrid encryption works in practice.

Bibliography

- [1] Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice* (7th ed.). Pearson Education.
- [2] National Institute of Standards and Technology. (2001). *Advanced Encryption Standard (AES)*. FIPS Publication 197.
- [3] Rivest, R. L., Shamir, A., & Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2), 120–126.
- [4] PyCryptodome Documentation. (2023). *PyCryptodome: A self-contained Python package of low-level cryptographic primitives*. Retrieved from <https://pycryptodome.readthedocs.io>
- [5] Löthberg, T. (2023). *CustomTkinter: A modern and customizable python UI-library*. Retrieved from <https://github.com/TomSchimansky/CustomTkinter>
- [6] Cross Crypto TS. (2025). *Cross Crypto TS: Hybrid encryption library (AES-GCM + RSA-OAEP) for TypeScript, Python, and Rust*. Retrieved from <https://www.npmjs.com/package/cross-crypto-ts>
- [7] Akshayaa, B. K. (2024). *Dual-Encryption Integrative RSA-AES Encryption for File Protection*. GitHub Repository. Retrieved from <https://github.com/Akshayaa-bk/Dual-Encryption-Integrative-RSA-AES-Encryption-for-File-Protection>
- [8] Alexylon. (2022). *Ferrocrypt: Rust library, CLI, and desktop application for file encryption*. GitHub Repository. Retrieved from <https://github.com/alexylon/Ferrocrypt>
- [9] Petrucci, V. (2025). *GhostLock: Hybrid Post-Quantum Encryption Protocol (X25519 + Kyber768 + ChaCha20-Poly1305)*. IETF Internet-Draft. Retrieved from <https://github.com/pacificitsolutions/ghostlock>
- [10] Chaubey, A. (2026). *ciph: Streaming file-encryption tool for large media files*. PyPI. Retrieved from <https://pypi.org/project/ciph>